

From Finite-Game Operators to Dense Numerical Linear Algebra

A computational account of the maintained potential–harmonic decomposition

Davide Legacci

July 2026

Abstract

This note documents the computational realization of the finite-game decomposition implemented in `CandoganDecomposition/decomposition.py`. It does not reproduce the structural theory of potential, harmonic, and nonstrategic games developed by Candogan et al. or its weighted formulation by Abdou et al. Instead, it gives the complete bridge from the abstract operators used by that theory to the arrays and dense linear-algebra operations executed by the program. We define the profile and edge indexings, derive every stored matrix, identify the sole Moore–Penrose pseudoinverse formed by the maintained implementation, and show how normalization and the exact-flow projection are applied without building their full matrices. We then derive time and storage bounds in terms of the number of players, profiles, and response edges. Finally, a reproducible benchmark on weighted random games measures geometry construction and cached decomposition separately. The data expose the expected exponential scaling in the number of players and the practical memory cost of storing dense incidence and gradient matrices.

Contents

1	From abstract operators to program operations	2
1.1	Scope and conventions	2
1.2	The oriented response graph	3
1.3	Weighted profile and flow spaces	3
1.4	The game-flow operator without a game-flow matrix	4
1.5	Normalization without a normalization matrix	4
1.6	The weighted Laplacian and its pseudoinverse	5
1.7	Returning from flows to payoff tensors	6
1.8	Complete program pipeline	7
2	Complexity analysis	7
2.1	Size parameters	7
2.2	One-time geometry construction	8
2.3	Cost of one decomposition after caching	9

2.4	Equal action counts and exponential growth	9
2.5	Computational implications	9
3	Reproducible timing experiments	10
3.1	Protocol	10
3.2	Results	10
3.3	Numerical validation and interpretation	13
3.4	Reproduction	13

1 From abstract operators to program operations

1.1 Scope and conventions

The theory is taken from Candogan et al. [1] in the uniform case and Abdou et al. [2] in the weighted case. Only the formulas needed to identify a theoretical object with a concrete program operation are recalled. In particular, no attempt is made to reprove the decomposition theorem.

Let there be $n \geq 1$ players. Player i has the finite action set

$$A_i = \{0, 1, \dots, d_i - 1\}, \quad d_i \geq 1,$$

and the pure-profile set is

$$A = \prod_{i=1}^n A_i, \quad M = |A| = \prod_{i=1}^n d_i.$$

The input measures are vectors

$$\mu_i = (\mu_i(0), \dots, \mu_i(d_i - 1)), \quad \mu_i(a_i) > 0.$$

Strict positivity is checked by the constructor. If no measures are supplied, the program sets every entry to one. Measures are not normalized. Define

$$m_i = \sum_{a_i \in A_i} \mu_i(a_i), \quad p(a) = \prod_{i=1}^n \mu_i(a_i), \quad \mu_{-i}(a_{-i}) = \prod_{j \neq i} \mu_j(a_j).$$

The maintained interface does not accept a separate opponent comeasure γ ; the implemented convention is $\gamma_i \equiv 1$.

A game is represented by a real tensor

$$u \in \mathbb{R}^{n \times d_1 \times \dots \times d_n},$$

where $u_i(a)$ is stored at `payoffs[i, a1, ..., an]` after converting the mathematical player index to Python's zero-based index. A flat vector of length nM is also accepted and reshaped in NumPy C order. In the formulas below, the profile index is

$$\kappa(a_1, \dots, a_n) = \sum_{i=1}^n a_i \prod_{j=i+1}^n d_j \in \{0, \dots, M - 1\}.$$

This is NumPy C-order indexing. The program uses `np.ndindex` to enumerate profiles, `reshape` to interpret flat payoffs, and `np.ravel_multi_index` to map profile tuples to columns.

1.2 The oriented response graph

Two profiles are response adjacent for player i when they agree in every coordinate except coordinate i . The code stores one orientation of every undirected response edge. For each opponent profile a_{-i} and each pair $0 \leq r < s < d_i$, it stores

$$e = (i, a^-, a^+), \quad a^- = (r, a_{-i}), \quad a^+ = (s, a_{-i}),$$

with the entries inserted in their correct coordinate positions. The number of player- i edges and total edges are therefore

$$E_i = \frac{M}{d_i} \binom{d_i}{2} = \frac{M(d_i - 1)}{2}, \quad E = \sum_{i=1}^n E_i = \frac{M}{2} \sum_{i=1}^n (d_i - 1). \quad (1)$$

Every two profiles can be connected by changing their coordinates one at a time, so the response graph is connected, including the trivial one-vertex case.

Fix the program's edge ordering. Its dense incidence matrix $B \in \mathbb{R}^{E \times M}$ is

$$B_{e,c} = \begin{cases} -1, & c = \kappa(a^-), \\ +1, & c = \kappa(a^+), \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Thus each row contains exactly two nonzero entries, although the implementation stores all EM entries as double-precision numbers. The arrays `edge_players`, `edge_tails`, and `edge_heads` store the corresponding integer metadata.

1.3 Weighted profile and flow spaces

Define the diagonal profile-weight matrix

$$P = \text{diag}(p(a) : a \in A) \in \mathbb{R}^{M \times M}.$$

For an edge $e = (i, a^-, a^+)$ define

$$s_e = \mu_{-i}(a_{-i})^{-1/2}, \quad q_e = p(a^-)p(a^+).$$

Let

$$S = \text{diag}(s_e : e \in E), \quad Q = \text{diag}(q_e : e \in E).$$

The weighted gradient matrix is

$$G = SB \in \mathbb{R}^{E \times M}. \quad (3)$$

Consequently, for $f \in \mathbb{R}^M$,

$$(Gf)_e = s_e(f(a^+) - f(a^-)).$$

The program stores the diagonal entries of P , Q , and S as the vectors `profile_weights`, `flow_weights`, and `gradient_scales`. It does not allocate the three diagonal matrices. Multiplication by a diagonal matrix is implemented as elementwise vector multiplication. It does allocate the dense matrix G as `gradient_matrix`.

The two inner products are

$$\langle f, g \rangle_0 = f^T P g, \quad \langle x, y \rangle_1 = x^T Q y. \quad (4)$$

Because all measures are positive, P and Q are positive definite. The adjoint of G with respect to (4) is therefore

$$G^* = P^{-1}G^TQ. \quad (5)$$

The method `codifferential(x)` executes exactly three operations: elementwise multiplication Qx , dense multiplication $G^T(Qx)$, and elementwise division by the diagonal of P .

1.4 The game-flow operator without a game-flow matrix

If player-major payoffs are flattened into $\mathbb{R}^{nM}$, the game-flow map could be represented by a matrix $D \in \mathbb{R}^{E \times nM}$. Its row for $e = (i, a^-, a^+)$ would have only two nonzero entries,

$$D_{e,(i,a^-)} = -s_e, \quad D_{e,(i,a^+)} = s_e.$$

Hence

$$(Du)_e = s_e(u_i(a^+) - u_i(a^-)). \quad (6)$$

The maintained program does not form D . It gathers the two payoff arrays using the integer metadata arrays `edge_players`, `edge_tails`, and `edge_heads`; it then subtracts them and multiplies by `gradient_scales`. This computes (6) in $\mathcal{O}(E)$ arithmetic and storage rather than allocating an $E \times nM$ matrix.

The following identity shows explicitly that the implemented codifferential recovers the weighted harmonic balance operator.

Proposition 1. *For every game u and profile a ,*

$$(G^*Du)(a) = \sum_{i=1}^n \sum_{b_i \in A_i} \mu_i(b_i) [u_i(a) - u_i(b_i, a_{-i})]. \quad (7)$$

Proof. Consider the unoriented edge joining a to (b_i, a_{-i}) . Incidence signs in G^T and in the payoff difference in Du combine so that its contribution at vertex a is

$$q_e s_e^2 [u_i(a) - u_i(b_i, a_{-i})].$$

Since

$$q_e s_e^2 = p(a)p(b_i, a_{-i})\mu_{-i}(a_{-i})^{-1} = p(a)\mu_i(b_i),$$

division by $p(a)$ in (5) gives the corresponding term in (7). Summing over players and alternatives proves the identity. The term $b_i = a_i$ is zero and may be included without changing the sum. $\square$

1.5 Normalization without a normalization matrix

For each player i , define the conditional weighted-average operator

$$(\Lambda_i f)(a_i, a_{-i}) = \frac{1}{m_i} \sum_{b_i \in A_i} \mu_i(b_i) f(b_i, a_{-i}), \quad (8)$$

whose value is constant in a_i , and let $\Pi_i = I - \Lambda_i$. On games these operators act player by player:

$$(\Lambda u)_i = \Lambda_i u_i, \quad (\Pi u)_i = \Pi_i u_i.$$

The code applies (8) by `np.tensordot(mu[i], array, axes=((0,), (i,)))` divided by m_i , then expands the removed axis for broadcasting. No $nM \times nM$ normalization matrix is built.

Lemma 1. Λ_i and Π_i are complementary orthogonal projections for the weighted profile inner product. Moreover,

$$\sum_{a_i \in A_i} \mu_i(a_i)(\Pi_i f)(a_i, a_{-i}) = 0 \quad \text{for every } a_{-i}. \quad (9)$$

Proof. Applying (8) twice does not change the conditional average, so $\Lambda_i^2 = \Lambda_i$ and $\Pi_i^2 = \Pi_i$. Equation (9) follows by expanding $\Pi_i = I - \Lambda_i$. If h is independent of a_i , then for each a_{-i} ,

$$\sum_{a_i} \mu_i(a_i)(\Pi_i f)(a_i, a_{-i})h(a_{-i}) = 0.$$

Multiplication by the positive factor $\mu_{-i}(a_{-i})$ and summation over a_{-i} proves orthogonality between the ranges of Π_i and Λ_i . $\square$

The program first computes

$$u^S = \Pi u, \quad u^N = u - u^S = \Lambda u. \quad (10)$$

Thus u^N is the chosen nonstrategic representative. Equation (6) also shows $Du^N = 0$. Conversely, $Du = 0$ forces $u_i(\cdot, a_{-i})$ to be constant on the complete action graph for every i and a_{-i} , so $\ker D$ is exactly the nonstrategic subspace.

1.6 The weighted Laplacian and its pseudoinverse

Given a flow $x \in \mathbb{R}^E$, the program finds its closest exact flow by solving

$$\min_{f \in \mathbb{R}^M} \|x - Gf\|_Q^2, \quad \|y\|_Q^2 = y^T Q y. \quad (11)$$

Differentiating the quadratic objective yields the normal equations

$$Lf = b, \quad L = G^T Q G, \quad b = G^T Q x. \quad (12)$$

The symmetric positive-semidefinite matrix $L \in \mathbb{R}^{M \times M}$ is the weighted stiffness form of the graph Laplacian. The operator $G^* G$ is $P^{-1} L$; the code forms L , not $P^{-1} L$, because (12) is the symmetric system relevant to the least-squares projection.

The constructor computes

$$L = G^T (Q G) \quad (13)$$

using dense matrix multiplication and then stores

$$L^\dagger = \text{pinv}(L) \quad (14)$$

using `np.linalg.pinv(..., hermitian=True)`. This is the only Moore–Penrose pseudoinverse built by the maintained numerical core. The Hermitian flag permits NumPy to use the symmetric eigensolver path. Numerical rank is determined by NumPy’s floating-point singular-value cutoff.

Because the response graph is connected and all weights are positive,

$$\ker G = \ker L = \text{span}\{\mathbf{1}\}.$$

Moreover, $b = G^T Q x$ is orthogonal to $\mathbf{1}$ in the Euclidean inner product, since $G\mathbf{1} = 0$, and therefore $b \in \text{range}(L)$. The program first computes the minimum-Euclidean-norm solution

$$\tilde{\phi} = L^\dagger b$$

and then fixes the desired weighted gauge:

$$\phi = \tilde{\phi} - \frac{\mathbf{1}^T P \tilde{\phi}}{\mathbf{1}^T P \mathbf{1}} \mathbf{1}. \quad (15)$$

Since $L\mathbf{1} = 0$, gauge adjustment (15) preserves the normal equations and does not change $G\phi$. It enforces

$$\sum_{a \in A} p(a) \phi(a) = 0.$$

The exact and co-closed flow components are then

$$x^{\text{ex}} = G\phi, \quad x^{\text{cc}} = x - G\phi. \quad (16)$$

Substituting (12) into (5) gives $G^* x^{\text{cc}} = 0$. Also, for any f ,

$$\langle x^{\text{cc}}, Gf \rangle_1 = f^T G^T Q x^{\text{cc}} = 0,$$

so (16) is the Q -orthogonal projection.

The formal exact-flow projector is

$$\mathcal{P}_{\text{ex}} = GL^\dagger G^T Q \in \mathbb{R}^{E \times E}. \quad (17)$$

The implementation never constructs (17). It applies its four factors successively to one flow, reducing the persistent dimension of the pseudoinverse from $E \times E$ to $M \times M$.

1.7 Returning from flows to payoff tensors

Apply the preceding flow calculation to

$$x = Du^{\text{S}}.$$

The normalized potential game induced by ϕ is computed player by player:

$$u_i^{\text{P}} = \Pi_i \phi. \quad (18)$$

The conditional average removed from ϕ is independent of player i 's own action. It therefore cancels across every player- i response edge, and (6) implies

$$Du^{\text{P}} = G\phi = x^{\text{ex}}. \quad (19)$$

Finally, the harmonic payoff tensor is formed by subtraction:

$$u^{\text{H}} = u^{\text{S}} - u^{\text{P}}. \quad (20)$$

Equations (10) and (20) give reconstruction identically,

$$u = u^{\text{N}} + u^{\text{P}} + u^{\text{H}}. \quad (21)$$

Equations (19) and (16) give

$$Du^{\text{H}} = x^{\text{cc}}, \quad G^* Du^{\text{H}} = 0.$$

The interpretation of this normalized, co-closed residual as the harmonic component is the theoretical result imported from [1, 2]; the program's role is to realize the associated orthogonal projections.

The game-space inner product used for reported norms is

$$\langle u, v \rangle_{\text{game}} = \sum_{i=1}^n m_i \sum_{a \in A} p(a) u_i(a) v_i(a). \quad (22)$$

It is implemented by looping over players and applying one weighted dot product per player. For normalized games, D is an isometry. Indeed, for fixed i and a_{-i} ,

$$\begin{aligned} & \sum_{r < s} \mu_i(r) \mu_i(s) (u_i(s, a_{-i}) - u_i(r, a_{-i}))^2 \\ &= m_i \sum_r \mu_i(r) u_i(r, a_{-i})^2 - \left(\sum_r \mu_i(r) u_i(r, a_{-i}) \right)^2. \end{aligned}$$

The final square vanishes under (9); multiplying by $\mu_{-i}(a_{-i})$ and summing proves

$$\|Du\|_Q^2 = \|u\|_{\text{game}}^2 \quad \text{for normalized } u.$$

Thus flow orthogonality transfers to game orthogonality. The reported potentialness is

$$\rho(u) = \frac{\|u^P\|_{\text{game}}}{\|u^P\|_{\text{game}} + \|u^H\|_{\text{game}}}, \quad (23)$$

with NaN returned when both norms vanish.

1.8 Complete program pipeline

For a fixed skeleton and measure, **GameGeometry** performs the following one-time construction:

1. validate (d_i) and (μ_i) ; compute m_i , profiles, and $p(a)$;
2. enumerate oriented response edges and their player, tail, and head indices;
3. allocate B , compute the vectors (s_e) and (q_e) , and allocate $G = SB$;
4. form $L = G^T Q G$ and store $L^\dagger$.

For each payoff tensor, **decompose** then executes:

1. $u^S = \Pi u$ and $u^N = u - u^S$;
2. $x = Du^S$ by indexed payoff differences;
3. $b = G^T Q x$, $\tilde{\phi} = L^\dagger b$, and the gauge correction (15);
4. $x^{\text{ex}} = G\phi$ and $x^{\text{cc}} = x - x^{\text{ex}}$;
5. $u_i^P = \Pi_i \phi$ and $u^H = u^S - u^P$;
6. weighted game norms and potentialness (23).

Table 1 gives the direct object-to-code correspondence.

Remark 1 (Finite precision). *All maintained numerical arrays are converted to double precision. Exact orthogonality and co-closedness therefore appear as residuals near machine precision. The symbolic module under **CandoganDecomposition/research** constructs exact operators for small games, but it is separate from the floating-point decomposition documented here.*

2 Complexity analysis

2.1 Size parameters

The input payoff tensor contains nM scalars. The response graph contains E edges given exactly by (1). Complexity must be stated in both M and E : two games with the same

Table 1: Mathematical objects and their concrete implementation.

Object	Implementation
A, κ	<code>profiles</code> ; NumPy C-order reshape and ravel indexing
oriented edges	<code>edges, edge_players, edge_tails, edge_heads</code>
P	diagonal stored as <code>profile_weights</code>
Q	diagonal stored as <code>flow_weights</code>
S	diagonal stored as <code>gradient_scales</code>
B	dense <code>incidence_matrix</code>
$G = SB$	dense <code>gradient_matrix</code>
D	not stored; <code>game_flow</code> uses indexed differences
G^*	not stored; <code>codifferential</code> applies $P^{-1}G^TQ$
Λ_i, Π_i	not stored; <code>normalize_profile</code> uses tensor contraction
$L^\dagger$	dense <code>_weighted_laplacian_pinv</code>
$\mathcal{P}_{\text{ex}}$	not stored; applied as $GL^\dagger G^TQ$
u^N, u^P, u^H	arrays in the returned <code>Decomposition</code> record

number of profiles may have very different edge counts. For example, a 32×32 two-player game and a ten-player binary-action game both have $M = 1024$, but their edge counts are 31744 and 5120, respectively.

The current implementation uses dense matrices even though every incidence row has two nonzeros. The following analysis describes the program as written, not an ideal sparse reformulation.

2.2 One-time geometry construction

Profile enumeration creates M tuples of length n , requiring $\mathcal{O}(nM)$ Python-level work. Edge enumeration creates E records containing profiles of length n , requiring $\mathcal{O}(nE)$ Python-level work and metadata.

Allocating the zero-filled dense incidence matrix costs $\Theta(EM)$ writes; placing its $2E$ nonzeros is lower order. Computing all gradient scales costs $\mathcal{O}(nE)$ scalar operations. Forming $G = SB$ touches every dense entry and costs $\Theta(EM)$.

The dense product

$$L = G^T(QG)$$

multiplies an $M \times E$ matrix by an $E \times M$ matrix. Under classical dense arithmetic this costs $\mathcal{O}(EM^2)$ operations. The symmetric pseudoinverse of the $M \times M$ matrix costs $\mathcal{O}(M^3)$ operations and $\mathcal{O}(M^2)$ workspace. Thus the setup bound is

$$T_{\text{setup}} = \mathcal{O}(nM + nE + EM^2 + M^3). \quad (24)$$

Optimized BLAS and symmetric eigensolvers change constants and hardware utilization but not these dense dimension counts.

The three leading persistent floating-point arrays are B , G , and $L^\dagger$. With eight-byte doubles, they occupy exactly

$$8(2EM + M^2) \text{ bytes}. \quad (25)$$

The measured persistent NumPy storage also includes weight and index vectors, while Python edge objects add $\mathcal{O}(nE)$ implementation-dependent overhead. Peak construction memory is larger than (25): the expression QG creates an additional $E \times M$ temporary, L occupies M^2 entries, and the eigensolver requires workspace.

2.3 Cost of one decomposition after caching

Once $L^\dagger$ is cached, normalization costs $\mathcal{O}(nM)$, and the indexed game-flow operation costs $\mathcal{O}(E)$. Applying G^T to a flow and G to a potential costs $\mathcal{O}(EM)$ each because G is dense. Applying $L^\dagger$ to the right-hand side costs $\mathcal{O}(M^2)$. Norms, reconstruction, and output assembly contribute $\mathcal{O}(nM + E)$. Therefore

$$T_{\text{cached}} = \mathcal{O}(nM + EM + M^2). \quad (26)$$

The returned arrays use $\mathcal{O}(nM + E + M)$ storage in addition to the cached geometry. Calling the convenience function `decompose(payoffs, mu)` constructs a new geometry and pays (24) every time. Reusing a `GameGeometry` instance is therefore essential for a sequence of games with the same skeleton and measures.

2.4 Equal action counts and exponential growth

Suppose every player has $q \geq 2$ actions. Then

$$M = q^n, \quad E = \frac{n(q-1)}{2}q^n. \quad (27)$$

Substitution into (24), (25), and (26) gives

$$T_{\text{setup}} = \mathcal{O}((n(q-1) + 1)q^{3n}), \quad (28)$$

$$S_{\text{persistent,lead}} = 8q^{2n}(n(q-1) + 1) \text{ bytes}, \quad (29)$$

$$T_{\text{cached}} = \mathcal{O}((n(q-1) + 1)q^{2n}). \quad (30)$$

The dependence on the number of players is exponential because the normal-form input itself has q^n profiles. The dense implementation adds another factor of approximately q^n in persistent matrix storage and two additional factors of profile dimension in geometry construction.

For fixed $n = 2$ and growing q , one has $M = q^2$ and $E = q^2(q-1) = \Theta(q^3)$. Consequently, the leading dense Gram product is $\mathcal{O}(q^7)$, persistent storage is $\Theta(q^5)$, and cached decomposition is $\mathcal{O}(q^5)$. For fixed $q = 2$ and growing n , $M = 2^n$ and $E = n2^{n-1}$; setup is $\mathcal{O}(n2^{3n})$, leading storage is $\Theta(n2^{2n})$, and cached decomposition is $\mathcal{O}(n2^{2n})$.

Table 2: Leading costs of the maintained dense implementation.

Operation	Time	Persistent numerical storage
Profile and edge metadata	$\mathcal{O}(nM + nE)$	$\mathcal{O}(nM + nE)$ objects/indices
Dense B and G	$\Theta(EM)$ initialization	16 EM bytes
$L = G^T QG$	$\mathcal{O}(EM^2)$	transient $EM + M^2$ doubles
$L^\dagger$	$\mathcal{O}(M^3)$	8 M^2 bytes plus workspace
Cached decomposition	$\mathcal{O}(nM + EM + M^2)$	$\mathcal{O}(nM + E + M)$ output

2.5 Computational implications

The mathematical incidence and gradient operators are sparse, but the current arrays are dense. A sparse incidence representation would reduce storage for B and G from $\Theta(EM)$ to $\Theta(E)$ and reduce their matrix-vector products to $\Theta(E)$. It would not by itself remove the cost

of a dense $L^\dagger$. Avoiding that matrix would require solving the constrained weighted Laplacian system for each right-hand side, for example by a sparse factorization or iterative method with an explicit gauge constraint. Such a solver would trade the present expensive setup and cheap reuse pattern for a different factorization/solve tradeoff. These are prospective improvements; the bounds above describe only the committed implementation.

3 Reproducible timing experiments

3.1 Protocol

The benchmark scripts and raw results are under `CandoganDecomposition/Documentation/experiments`. Each successful case uses a deterministic random generator with base seed 20260715. Payoffs are independent standard normal values. Each action measure is independently drawn from the uniform distribution on $[0.5, 2]$, so every run exercises the weighted rather than only the uniform path. Matrix dimensions, and hence the complexity analyzed in Section 2, do not depend on the realized positive weights.

Two timings are recorded separately with `time.perf_counter_ns`:

1. *setup*: construction of a new `GameGeometry`, including edge enumeration, dense matrices, L , and $L^\dagger$;
2. *cached*: one call to `geometry.decompose(payloads)` after a warm-up call, with the same geometry and pseudoinverse reused.

Setup uses seven independent samples when the first construction is below 0.5 seconds, five below 2 seconds, three below 8 seconds, and one for larger cases. The table reports the median. Cached timings use nine batches; the number of decompositions per batch is doubled until a batch lasts at least 0.15 seconds, and the median per-call time is reported. BLAS-related thread environment variables are fixed to one.

The run was performed on a MacBook Pro with an Apple M1 Pro processor (eight cores: six performance and two efficiency) and 16 GB of memory. The benchmark interpreter was Python 3.12.3 reporting an x86_64 process, with NumPy 2.4.3, on macOS 26.5.1. Thus these values characterize this software/hardware combination and are not universal constants.

Before allocating a case, the script evaluates the leading persistent storage formula (25). Cases above 768 MiB are marked as skipped. This is a conservative operational stopping rule for the experiment, not a claim that 768 MiB is an intrinsic mathematical limit. It also does not bound all temporary allocations.

3.2 Results

Thirty of 34 configured cases were executed. The four cases skipped by the storage rule were 36×36 , 2^{12} , 3^8 , and 7^4 . Table 3 gives representative medians. “Storage” is the sum of the persistent NumPy arrays owned by the geometry object; it excludes Python object overhead and temporary arrays.

Figure 1 plots all successful runs against M . The families do not collapse onto a single curve because E is an independent size parameter. At $M = 1024$, for example, the 32×32 case has $E = 31744$, setup time 4.62 s, and cached time 81.6 ms, whereas the ten-player binary case has $E = 5120$, setup time 1.10 s, and cached time 10.1 ms. This comparison directly exhibits the E dependence in (24) and (26).

Table 3: Representative weighted-game benchmark results.

Skeleton	M	E	Setup (ms)	Cached (ms)	Storage (MiB)
2×2	4	4	0.301	0.174	0.00
2×3	6	9	0.311	0.173	0.00
2^3	8	12	0.431	0.209	0.00
4×4	16	48	0.641	0.173	0.02
3^3	27	81	0.989	0.209	0.04
2^6	64	192	3.039	0.354	0.23
10×10	100	900	8.477	0.218	1.48
4^4	256	1536	35.200	0.416	6.56
20×20	400	7600	210.867	3.375	47.90
5^4	625	5000	353.235	2.822	50.86
32×32	1024	31744	4617.605	81.621	505.22
6^4	1296	12960	3391.761	22.097	269.61
2^{11}	2048	11264	8798.432	42.857	384.45
3^7	2187	15309	12153.455	42.138	547.97

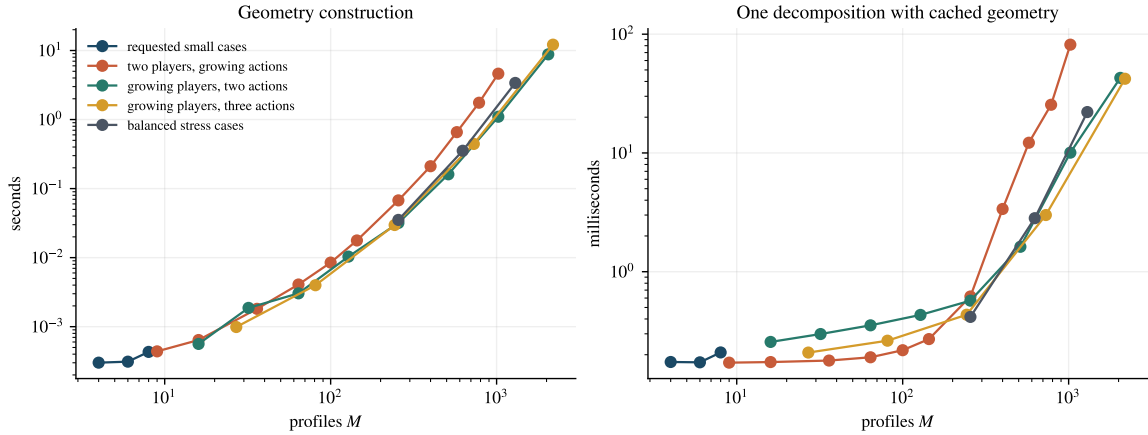


Figure 1: Geometry construction and cached decomposition versus the number of profiles. Both axes are logarithmic. Lines connect cases within a controlled family and are guides to the eye, not fitted complexity models.

Figure 2 isolates the two main growth experiments. For two players, setup grows from 0.64 ms at 4×4 to 4.62 s at 32×32 . For binary-action games, adding players from four to eleven increases M from 16 to 2048 and setup from 0.56 ms to 8.80 s. The logarithmic vertical scale makes the transition from fixed Python overhead on tiny cases to dense linear-algebra cost on larger cases visible.

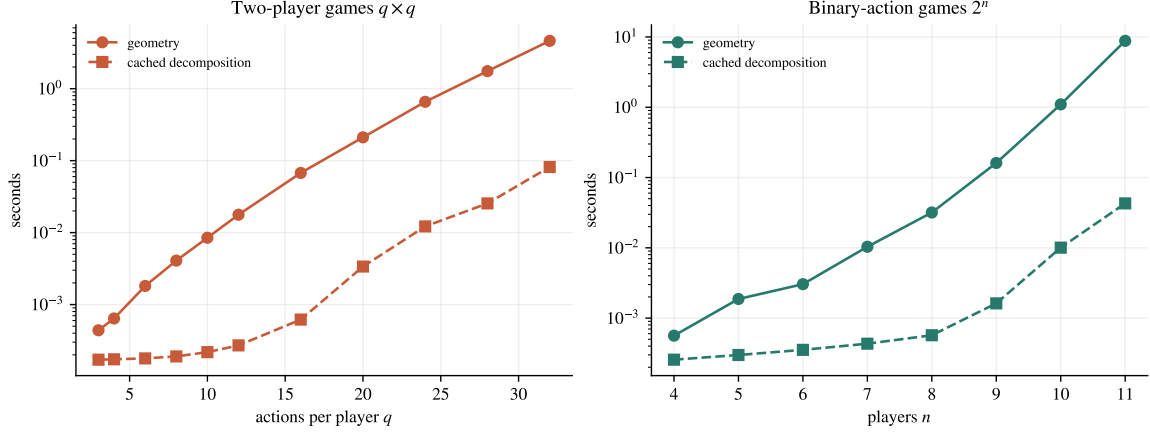


Figure 2: Controlled scaling with actions per player (left) and number of players (right). The vertical axes are logarithmic.

Persistent numerical storage follows the dimension formula closely, as shown in Figure 3. The largest executed case, 3^7 , stores about 548 MiB of NumPy arrays and requires 12.15 s for its single measured geometry construction. The 3^8 successor would require approximately 5.45 GiB for the leading persistent arrays alone and was therefore not allocated. This is the practical stopping point of the present dense experiment.

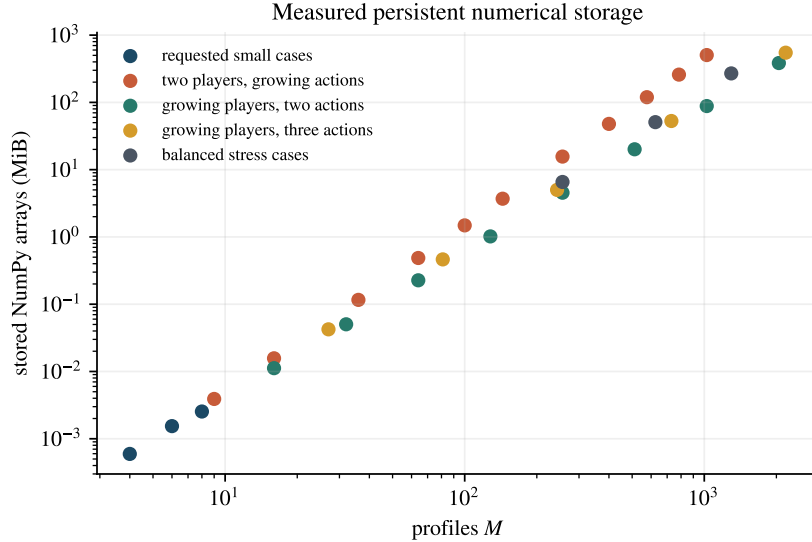


Figure 3: Persistent NumPy storage owned by each constructed geometry. Both axes are logarithmic; temporary arrays and Python object overhead are not included.

3.3 Numerical validation and interpretation

For every timed game, the script recomputes a result and records three residuals. Across the 30 successful cases, the maximum reconstruction residual

$$\|u - u^N - u^P - u^H\|_\infty$$

was exactly zero in floating-point arithmetic because the final component is formed by subtraction. The largest co-closedness residual

$$\|G^* D u^H\|_\infty$$

was 1.06×10^{-12} , and the largest absolute potential–harmonic game inner product was 7.67×10^{-12} . These values are consistent with double-precision eigensolver and matrix-multiplication error at the tested dimensions.

The data support three computational conclusions.

1. Geometry construction is the dominant cost because it forms a dense Gram matrix and its pseudoinverse. Caching changes the relevant bound from (24) to (26).
2. Profile count alone is insufficient to predict runtime: edge count explains substantial differences between skeletons with the same M .
3. Persistent memory, rather than the time for one cached decomposition, becomes the first hard barrier in the present dense design. A sparse operator and a solve-based treatment of the Laplacian are the natural route to larger games.

The experiment is descriptive rather than a proof of asymptotic exponents; the rigorous exponents are those derived from the executed matrix dimensions in Section 2.

3.4 Reproduction

From the repository root, the committed data and figures can be regenerated with

```
cd CandoganDecomposition/Documentation/experiments
python benchmark_decomposition.py
python plot_benchmarks.py
```

The first command overwrites `benchmark_results.csv` and `benchmark_metadata.json`; the second overwrites the PDF/PNG figures and `benchmark_table.tex`. Timing changes across runs are expected, but the case definitions, random inputs, matrix dimensions, and numerical residual checks are deterministic.

Implementation version

The implementation analyzed here is the maintained NumPy core as present in the repository in July 2026. Its source is `CandoganDecomposition/decomposition.py`. The benchmark data were collected on July 15, 2026. The document and experiment directory are intended to evolve together if the core is later changed to sparse storage or a solve-based projection.

References

- [1] O. Candogan, I. Menache, A. Ozdaglar, and P. A. Parrilo, Flows and decompositions of games: harmonic and potential games, *Mathematics of Operations Research* 36 (2011), 474–503. <https://doi.org/10.1287/moor.1110.0500>.
- [2] J. Abdou, N. Pnevmatikos, M. Scarsini, and X. Venel, Decomposition of games: some strategic considerations, *Mathematics of Operations Research* 47 (2022), 1766–1784.